

Using the am1 macro assembler for the PDP-1

This document describes the **am1** macro assembler and how to use it.

This is version 1.17 and covers up through am1 version 1.17; it will be updated as needed.

Edit date 15-Feb-2026

What is am1?

(Yet) Another Macro assmblr for the PDP-1.

Why do we need another one? Simply because the avaiable ones seem to essentially dupcate the old **macro1** assembler with minor tweaks. It's time for a modern assembler that brings productive features and better error checking and better readability.

Am1 provides some significant new features while maintaining some compatibility with existing macro sources. Additionally, a separate conversion tool, **macttoam1**, is provided to process an existing macro source file and convert it to **am1** syntax.

It can, in fact, produce as its output valid source that can be assembled by **macro1**.

The features and differences

First, two different assembly modes are supported, generating of **macro1** source and direct generation of rim-lodable binary code. While most of the language constructs apply to both, no extended memory features such as *bank* and *bank references*, are supported by **macro1**, and neither are the PDP-1D extended instructions.

Code will still be generated but should just be for reference as any code in extended memory will just overlay code in bank 0, and the PDP-1D and special instructions will give an error unless defined in *macro1**.

Features

- Uses **cpp** to bring the full power of **cpp** to the code
- Provides local variables and local scoping
- Provides a new, explicit, variable support with initializers
- Allows easy use of ascii characters and strings
- Supports octal, decimal, and hex numbers without special statements
- Adds operators for and, or, xor, complement, multiply, divide, mod in expressions
- Adds the need-completion flag, C
- Adds sdb for Type 33 Symbol Generator support
- Adds dpyc, shorthand for *dpy-i C*
- Can select between keeping -0 or automatically converting to +0 in expressions
- Allows space-separated symbols to be treated as $a \mid b \mid c$ instead of $a + b + c$
- Adds parenthesized expressions
- Supports extended memory

- Provides symbol exports and imports for linking programs together
- Can share location symbols across memory banks
- Can generate either **macro1** source or rim format binary code output
- The rim format loads a new loader to support extended memory use
- Does **not** allow redefining symbols (but cpp defines can)
- Does **not** treat tab as a statement delimiter
- 'Punches' the readable program header line in binary mode, like the original **macro** does

Differences in source code

These changes were done to allow the above changes and to remove some of the ambiguous use of characters in **macro1** that were poorly handled.

- Constants are now of the form `[xxx` and `[xxx]`, not `(xxx` and `(xxx)`
- Multiple statemens on a line are separated by `;` not tab
- Tabs are not statement delimiters and are treated as a space
- Cpp directives of the form `#xxx` are supported
- The operators `&` `|` `~` `^` `*` `/` `%%` (mod), as well as `(expr)` are added
- C is defined as 040000 for use with IOTS
- `dpyc` is defined as 724007 for convenience
- `sdb` is defined as 722007 for Type 33 support, equivalent to dpy-i if Type 33 not configured
- `%` is added to indicate a local variable, `%xxx`
- The numeric prefixes `0x`, `0d`, `0o` are added to mean hex, decimal, octal regardless of the current radix
- `'x'` is allowed to mean the value of a single ascii character, the usual escapes are recognized
- `ascii "xxx"` is provided to create a table of ascii character values similar to `text`
- The directives `local`, `addlocal`, and `endlocal` are provided to control local scoping
- The directive `bank` is provided to control placement in extended memory
- Location symbols in one bank can be referenced from another bank by using the `:banknum` suffix
- Explicit variable declarations with optional initializers is added, `var name=expr, name...`
- Tables with optional initializers are added, `table 10, 7+5`
- Macro1 style define-terminate blocks are **not** supported, use **cpp** `#defines`

A note on macro vs macro1

No particular effort is made to remain compatible with the original native **macro**, although if symbol lengths are limited to be unique in the first 3 characters and operators added in **macro1** aren't used, the generated code should be compatible.

Compatibility with **macro1** is provided as long as symbols are unique within the first 6 characters.

A note about **macro1** *define* and *terminate* constructs - as noted, these are not supported. But, see next.

But what about existing macro1 sources?

Never fear, help is here! - *Dudley Do-right*

The **mactoam1** utility, see its documentation, will convert existing source code to **am1** with good reliability, including converting **macro1** defines into the equivalent **cpp** defines.

If you have only a binary image, then use the **disassemble_tape** utility in its macro source mode to create a macro source file you can then process as above.

And of course, you can go back to macro sources from **am1** by using the macro source output flag.

Flex, flexo, FIO-DEC, concise???

While the native character set used in the PDP-1 is frequently called *flex* or *flexo*, it really isn't. Flexo is FIO-DEC, a 6 bit character code with an 8th bit of parity. But what is really used is *concise*, which is just FIO-DEC without the parity bit, resulting in a strictly 6 bit character.

You will see all these variations of naming used in PDP-1 documentation, but just remember, it's all just 6 bit *concise*. Unless it isn't, just to keep things confusing.

In this document, no matter what it's called, it's *concise*.

What goes on inside?

Unlike other PDP-1 assemblers, this is built with modern(ish) tools. It uses lex and yacc (flex and bison) for the lexical analysis and parsing. The result of that is a parse tree representing the program.

Having a parse tree means it's easy to plug in different code generators. Just walk the tree and emit what you want.

Locals are implemented with a local symbol table that is created when a local scope begins. It's pushed on a stack of local tables if a nested local scope is encountered. Each ending of a scope updates all the scope's local symbols to reflect the value of the pc where they are allocated and pops back to the next higher context, if any.

All references within the scope are then emitted as pc-relative so no externally visible symbols are created.

Constants are handled specially to allow for constant reuse. When a constant is encountered, its defining expression is used to create a hash code that is saved as the key in the constants symbol table. Thus, the next time the same value is seen the existing entry will be used. When a *constants* directive is seen, all constants added since the last instance of the directive are assigned locations starting from the current location, which will be updated to one past the last constant processed. However, the actual value is not computed until the end of the program so that forward references can be used.

If there are constants that have not been processed when the end of the program is reached, they will be automatically processed and placed just after the last location used. This might or might not be what is intended, so explicitly placing them is advised.

Each memory bank has its own context that holds the global symbols and constants, the current location, etc. The context is created the first time a bank is specified by the *bank* directive. The contexts are automatically switched when banks are switched.

A cross-bank reference looks up the given symbol in the global symbol table of the target bank and becomes the full 16-bit address if that symbol. If the symbol does not exist currently in the bank, it will be created. If it is never resolved in that bank, an error will be given at the end of the program.

Three code generators are implemented, one that emits correct macro1 code that can be assembled by it, and one

that emits binary suitable for rim loading, and one for generating listings. The parser side knows nothing about the details of the code generators, it just manages the symbol tables and creates the parse tree.

Wrapping the parsing and code generation is control logic that manages the files and optionally passes the input through **cpp**.

The companion tool, **mactool** handles the syntactic differences between **macro1** and **am1** and deals with translating them, and also understands the original *define-terminate* syntax and converts that to **cpp** *#define* statements. It is implemented using flex.

Building am1

Be sure you have flex and bison installed, then:

Just type make.

Usage

am1 [-Wabmlnsvz[xykp]] [-Dsymbol]... [-lpath]... [-ipath] sourcefile

- W don't print warnings
- a space means add, default is or
- b generate binary tape image code, the default action
- m generate **macro1** code
- l generate a program listing
- n don't run **cpp** on the input
- r don't output an initial rim loader
- s generate a symbol table, automatic if exports are done
- v print the version number and exit
- z replace -0 with 0 for math operation results
- Dsymbol define a symbol for **cpp**, -Dsym or -D sym are both accepted
- lpath add a search path to **cpp** for "files", -lpath or -l path are both accepted
- ipath set the root directory for searches, -ipath or -i path are both accepted

These additional flags are generally for debugging:

- x send lex debugging output to stderr
- y send yacc debugging output to stderr
- k keep intermediate **cpp** file
- p print the internal parse tree in readable form

Both **macro1** and binary code can be generated at the same time.

If warnings are disabled, errors will still be printed.

Both **macro** and **macro1** treat *a space b* as *a + b*, which leads to some tortuous expressions in code when multiple operands are combined. By default, **am1** treats a space as an *or* operation, which makes much more sense. However, the original behavior can be used via the *-a* flag.

The 1's complement -0 value, all bits set, when produced by a math operation in an expression is normally preserved. The -z flag overrides this and converts -0 to +0, all bits cleared. Again, this only affects the results of the binary math operators and unary minus, other values are not altered.

If the 'system' include files are not installed in the default location, /opt/pidp1-mods/Am1Includes, then the location should be specified either by using the -i *incroot* switch or by setting the environment variable *AM1INCDIR* to the full path to the location.

The -i flag has priority, followed by the environment variable, followed by the default.

At runtime, /usr/bin/cpp must exist if preprocessing is being done.

Listing file

The listing file is more complex than the **macro1** version in order to list code that is **#includeed**. Each file being processed will have an initial line identifying the file the listing following is from.

Multiple statements on a line will be listed on separate lines, but the line number will be the same for all.

Empty lines and comments do not show line numbers, only actual statements do so.

#define statements are not listed, but the expansion is.

While a listing can be produced for either macro or binary, the binary value for each location is only available if binary has been generated. For macro only output, the value field is meaningless. Of course, **macro1** will produce its own listing file, so creating one via **am1** is fairly useless other than seeing macro expansions.

Output files

All output files have the original file's name but with a different extension. The following can be produced:

- *file.rim* - binary output that can be read-in loaded
- *file.bin* - binary output that has no rim loader
- *file.mac* - text output that can (usually) be assembled by **macro1**
- *file.lst* - text output that is a listing of the assembled program, only generated for binary mode
- *file.cpp* - text output that is the intermediate output from the **cpp** preprocessor
- *file.sym* - text output that is a listing of the global symbols in the program

The sym file contains an initial line that is the filename of the original file followed by one line per symbol of the form:

```
aaaaaa F symbol-name
```

where *aaaaaa* is the full 16-bit address of the symbol's location in memory, and *F* is a flag, either *G*, *I*, or *X*, A *G* means the symbol wasn't exported, *X* means the symbol was, *I* symbols were imported.

Files with no loader

When binary code is being output and the -r command line flag is used, no initial rim loader will be written. This is useful for creating tapes that will be loaded after a program and is used in conjunction with the *stop* directive.

Additional tapes loaded must have been generated with the *-r* flag; if not, the results will be undefined.

Common uses are loading tables of data or loading additional already assembled code.

When used, a program that needs to load additional data or code should end with a *stop*. A no-loader tape can then be mounted and the *continue* switch pressed to load the tape. As long as tapes end with a *stop*, the process can be repeated.

If the program is to automatically start, the last tape should end with a *start* statement with the appropriate start address.

Using macros

The C preprocessor is used to handle macros, and you should be familiar with it.

The simple case:

```
#define foo bar
```

is essentially identical to the **macro1** *foo=bar*.

However, defining the equivalent of a macro is different. The preprocessor expects the complete definition to be one line. This is done by using line continuations and semicolons:

```
#define mymacro(a,b) lio a; cma; lac b
```

or

```
#define mymacro(a,b) \  
    lio a; \  
    cma; \  
    lac b
```

The backslash at the end of the line says 'ignore the end of the line, treat the next line as part of this line'. There is one non-obvious and somewhat annoying side-effect of this. Consider:

```
#define mymacro(a,b) \ // This is my macro  
    lio a; \  
    etc.
```

This will fail completely because of the line-joining. What this really looks like is:

```
#define mymacro(a,b) // This is my macro lio a;...
```

As you can see, everything following the comment appears as part of the comment! So, put your comments before the definition. Yes, this is annoying.

General program structure

All programs start with a title line. This is mandatory. If you forget it, then whatever the first line is in the source will be the title, probably not what was expected.

After the title line, any number of program lines are given, which can include standard C preprocessor directives such as `#include`, `#ifdef`, etc.

The assembler has a current location counter. This corresponds to the current memory address being used. It is initially set to location 4 and is changed as statements are assembled.

Program lines consist of *statements*, each of which is an *expression* or a *directive*, pseudo-instruction in **macro1** parlance terminated by a new line or a semicolon, `;`. Using a semicolon allows multiple statements on one line. A semicolon does not change the current line number but does change the current location counter. A new line changes both.

Comments are also allowed using the C notation:

```
// comment  
or  
statement.... // comment
```

Each statement generally increments the current location by one, but see more below. Comments and empty lines do not affect the current location.

Finally, the last line must be a *start xxx* statement to tell the loader where to start executing or a *stop* statement to allow for loading additional tapes before starting a program.

The original **macro1** was not particularly good at reporting errors, and a missing start would generally cause unintended behavior. This statement is now mandatory, enforced by the assembler.

Expressions

An *expression* is a sequence of *symbols*, *operators*, and *numbers*. Expressions are evaluated and the resulting value becomes the 18-bit result.

All the terms in an expression are evaluated at *assembly time* to give an integer value; *there is no computation done during the execution of an instruction!*.

The PDP-1 used 1's complement math, which no modern computers use. But, **am1** runs on a modern computer and uses 2's complement math. It automatically converts the results of math operations (`+`, `-`, `*`, `/`, `%`, and unary minus) to the 1's complement representation after computation, the values in the binary output will be the correct 1's complement values.

Remember that one annoying thing about 1's complement is that there are two values for zero, 0 and -0. 0 is all bits off, -0 is all bits on, and it is maintained in the output. For example, the operation `-1 + 1` results in -0, not 0 as you would normally expect.

Some of the math operations are not supported by **macro1**. These will be computed during assembly and emitted as numeric values.

Symbols

A *symbol* is a string of characters composed from an initial upper or lower case alphabetic character or underscore,

followed optionally by any number of alphanumeric characters, digits, and underscore e.g. *scratch_Location_1*.

Note that while **am1** allows effectively unlimited symbol lengths (1023), **macro** only allows 3 and **macro1** only allows 6. More precisely, symbols must be unique in the first 3 characters for **macro**, 6 for **macro1**.

Unlike **macro1**, a digit cannot be used as the first character of a symbol.

Symbols are of four general and one special types.

- location symbol e.g. *abc34*, which represents a named location in memory
- local symbol, e.g. *%mylocal*, which represents a location specifically within a local block
- reserved symbol, e.g. *lac*, generally an opcode
- variable
- constant symbol, e.g. [123]

Location symbols

An example of location symbols is:

```
    dac foo
    jmp bar
    lio zip:3
foo, 0
bar, .....
```

The first three are *references*, the value is the memory location assigned to each. The second two are *location assignments*. Each is given a memory location, which is the current program location.

The form for *zip* is a *fully-qualified* symbol that results in its full 16 bit address, see the section on *bank*, below.

It is an error to reference a symbol that never has a location assigned to it, and it is an error to define a location for the same symbol more than once in the same memory bank.

Note that this is a departure from **macro1** which allow arbitrary redefinition of *any* symbol, a bad idea.

A location symbol is *defined* when its name is first used. It is *resolved* when it is used as a location. In the example, the first use of *foo* is a definition, the second is the resolution. The second form is also called a *location assignment*.

Local symbols

Local symbols are a variation of location symbols. They are defined between *local* and *endlocal* directives and come in two forms, *predefined* and *ad-hoc*. Predefined locals are specified following the *local* statement while ad-hoc symbols are declared when used via a leading per-cent, %, symbol.

The two are distinct, predefined *a* and ad-hoc *%a* are separate symbols.

```

local a, b
%target, iot 31
a, cla
.
.
b,
jmp %target
endlocal

```

Additionally, more locals can be defined after the initial *local* directive by using *addlocal*.

```

local a, b
.
.
addlocal c, d.
...

```

These act in all respects just as if they were added in the original *local* directive.

Unlike **macro1** where all symbols are global and so all symbols must be unique or they will be redefined, local symbols allow code to be written that won't clash with other code. The local symbol exists only within the local block and the generated code uses pc-relative addressing to access it.

Local blocks can be nested up to 1024 deep. A local symbol in an enclosing block can be referenced within a nested block, and regular location symbols can be used and declared within local blocks.

Uf a local symbol is defined in a *local* or *addlocal* directive but never referenced, it will be ignored, no code will be generated for it.

Symbol exports and imports

A program can export its global symbols for other programs to use. This is useful for allowing separately assembled and loaded programs to interact. The exported symbols are in a *.sym* file, see above.

Symbols from other programs can be imported, making symbols in those programs accessible when loaded.

A program has control over exporting and importing symbols by using the *export* and *import* directives. See the directives *import* and *export*, below.

Variables

Variables are actually just a shorthand way of representing a common operation, which is naming a location in memory.

Variables must be explicitly declared via the *var* directive but then can be used just like any location symbol. A variable can have an optional initializer.

This:

```
var a, b=34+c
dac a
jmp b
```

and this:

```
    dac a
    jmp b
    ...
a, 0
b, 34+c
```

are exactly equivalent. Also see the *variables* directive below.

Constants

Constants are another shorthand, but with special behavior.

Examples of constants are:

```
lac [123]
lio [a+456  // a trailing comment
```

Functionally, the above is equivalent to:

```
    lac const1
    lio const2
    .
    .
const1, 123
const2, a+456
```

But, constants are also tracked and kept in a *constants pool*. Successive uses of the same constant all share one memory location for their value, saving space.

If a constant is the last thing on a line except for a trailing comment, the trailing] can be omitted.

Note that *last* means **last**, nothing but an optional trailing comment is allowed.

Best practice is to always close the constant.

Also see the *constants* directive, below.

Tables

Tables correspond to the **macro1** *dimension* pseudo-op. A table statement reserves a number of words, optionally initialized to a value. The table keyword is followed by an expression that must be able to be fully evaluated at that time.

Examples are:

```
table 10
table 5*7
```

An optional expression can follow which will be the value to fill the table with. If no value is given, the table will not be initialized:

```
table 10, 'z'
```

fills the table with the ascii value of the letter z.

The table keyword is a statement, and as such must be on a line by itself. This means that:

```
foo, table 5
```

is **not** legal.

Functionally, it is equivalent to the same number of the initializer value, e.g.

```
table 5, 17
-or-
17
17
17
17
17
17
```

Or, if there is no initializer:

```
table 5
-or-
.+5/
```

Using an initializer means that a 'tape' word will be written for each element of the table, which for large tables can significantly increase the size and load time.

Operators

The following operators are defined and listed in order of increasing priority.

The priority is the same as that for C and the operations are the same.

Operator	Meaning
	bitwise or
^	bitwise xor
&	bitwise and
<< >>	left-shift right-shift
+ -	addition subtraction
* / %%	multiply divide modulo
~	complement
-	unary minus, -n
()	expression nesting

Note that the *modulo* operator is as shown, double percents. This is because the percent symbol is already used for local variables. The doubling is to distinguish it from the beginning of a local symbol.

Internally, 2's complement arithmetic is used for most of the operators, but the result is adjusted to be a 1's complement value. The 1's complement -0 value, 7777777, can be produced by math operations such as -1+1, but by default will be converted to +0. This can be overridden if -0 is to be kept, see *Usage*.

Bitwise operations can of course affect any bit and their result is not adjusted, they are not math. If a -0 bit pattern results, it is kept as-is.

Numbers

Numbers are just that, an optional leading - followed by sequence of digits. However, the interpretation of the digitis can be either as octal or decimal numbers, depending upon the current radix, see *octal* and *decimal* below.

Several special representations for numbers are also provided, they override the current radix:

- 0oNNN an octal number, N must be 0-7
- 0dNNN a decimal number, N must be 0-9
- 0xNNN a hexadecimal number, N must be 0-9a-f or A-F

There are two additional special number representations:

- char cl, cm, or cr
- 'c'

The first is the same as the **macro1** version, *c* must be a valid *unshifted* Flex/Concise character, and the lmr specifies which 6 bit field in the resulting 18 bit value the character's value is placed.

The second results in the value of the ascii character, the usual escapes are allowed:

- \t tab
- \e escape
- \b backspace

- `\^c` control character `c`
- `\f` formfeed
- `\n` newline
- `\NNN` the value represented by the 3 **octal** digits
- `\\` backslash

Even though the above are interpreted in the base they specify, when emitted in the output, they will be given as the equivalent value in the current radix, octal or decimal. For example, if the current radix is octal, then `0x1f` will be emitted as `37`, the octal equivalent.

Directives

Directives, aka pseudo-instructions, tell the assembler to do special operations. Some generate code, some don't. In the list below, letters inside parentheses, `()`, are optional.

Directives are:

- location assignment, `xxx/`
- `loc(al)`
- `addloc(al)`
- `endloc(al)`
- `octal`
- `decimal`
- `flexo`
- `text`
- `ascii`
- `var`
- `variables`
- `constants`
- `bank`
- `start`
- `stop`
- `import`
- `export`
- two special directives, see below

Location assignment

Location assignments directly set the current location to the value given, which must be an expression that evaluates *at that time* to a value. This means that for any use of a symbolic location, that location must already be resolved. Using a forward reference actually makes no sense, and **macro1** doesn't allow it either.

Examples are:

```
100:    the location counter is set to 100
.+10:   the location counter is incremented by 10

a, foo
a+10:   the location counter is set to the location of foo plus 10, a must be defined *prior* to its use
```

Octal and decimal

These two set the current radix for numbers entered without an explicit radix specification. They stay in effect until the next one is seen. Any 'bare' number will be interpreted in that radix and an error given if the digits 8 or 9 are used in octal mode.

Examples are:

```
decimal
law 9   results in the octal word 700020

octal
law 9   will give an error
```

Regardless of the radix, the proper binary representation results.

Flexo

This packs up to 3 flex/concise characters into an 18-bit word. The first character is in the high 6 bits, the next in the middle 6 bits, the last in the low 6 bits. If not enough characters are given to fill the word, the space character, octal 0, is used.

NOTE: Remember that upper and lower case characters require a shift character, so something like

```
flexo Ok?
```

will not do what you expect. That sequence actually requires 6 characters, uppershift, O, lowershift, k, uppershift, ?. Unlike **macro1**, this is detected and a warning given.

Text

The **text** directive inserts a block of flex/concise characters packed 3 to a word. The location counter is incremented by the number of characters divided by 3, plus 1 if the number of characters wasn't a multiple of 3.

The string of characters is surrounded by a marker character, which can be any valid ascii character. The enclosed ascii characters are converted to flex/concise characters automatically.

Examples:

```
text "this is text"
text !This is text!
```

The first results in 4 words, 3 characters per word, packed first in the high 6 bits, second in the middle 6 bits, third in the low 6 bits. The location counter is advanced by 4.

The second results in 5 words because of the need to insert an upper-shift character and a lower-shift character. The location counter is advanced by 5.

Unlike the *ascii* directive, there is no end marker. You must know in your program how many words were used. Safe practice would be to put a marker word following the text with a value to indicate no more words. The flex/concise stop code, 013, is a good choice.

Ascii

The **ascii** directive inserts a block of ascii characters packed 2 to a word and always terminated by a binary 0 byte, as C strings are. The first character is in the high 9 bits of the word, the second in the low 9 bits. In each case, the high bit is not used and will always be 0.

The location counter is advanced by the number of characters divided by 2. If there are an even number of characters, one additional word is written and the location advanced by one more.

The string of characters is always surrounded by double-quotes, ", and the same escape sequences supported by the C number form are supported.

Examples:

```
ascii "this is text"
ascii "this is text!"
```

The first results in 7 words, 6 for the 12 characters and 1 for the terminating binary 0.

The second also results in 7 words, 6 plus the high 9 bits in the 7th word for the characters with the remainder of the 7th word binary 0.

In both cases the location counter is advanced by 7.

Variables and constants

These cause any declared variables or constants to be emitted at the current location. Subsequent declarations will be held until the next variables or constants constants.

If no directive is given, any variables and constants will be written at the end of the program, the location where the start or stop directive is given. This might not place these where you want, so using the directives is advised.

The location counter is updated appropriately after either directive.

Bank and cross-bank references

The bank directive causes all following code to be placed in the bank given, which must be between 0-31 decimal, 0-37 octal.

Each bank has its own global symbols and constants. These are preserved for each bank, so if a subsequent bank

command switches back to an earlier bank, all of its symbols and constants will still be valid. The current location in that bank is also saved, and will be the same as it was when a bank switch was done.

The first time a bank is switched to, **the current location will be set to 0.**

Remember to use the *constants* and *variables* directives in *each* bank where constants or variables are used.

Global location symbols in one bank can be referenced from another bank by using a *bank reference* modifier on a location symbol, *sym:bankno*, *sym:.*, or *sym:**, to reference a symbol.

When it is used, the value is the 16-bit address of that symbol.

The *sym:.* form means *in this current bank* and results in a full 16-bit address for the symbol.

For the *sym:bankno* form, if the location symbol is not defined in the target bank, it will be created. However, it must be resolved at some point in that bank or an error will be generated.

The *sym:** form is a wildcarded location. It means *whatever bank that symbol is defined in*. Since the same symbol name can be defined in multiple banks, a search rule is used. At the end of parse tree generation the location is resolved by searching the list of banks used from the first declared in a program by a *bank* statement to the last declared. The first match is the resolution.

However, if it is defined in the bank in which it is used, the symbol in that bank will be used and no searching done.

If not found in any bank, an error results.

For example:

```
100/  
eem  
  
bank 1  
200/  
lac a  
a, 0  
lio i [a:.]  
lac b:.*  
  
bank 0  
b, lac a:1  
lac a:*
```

In bank 0, a:1 will have the value 10201, as will a:.*.

In bank 1, a will have the value 201, its in-bank address, while a:. (a:dot) will have the value 10201.

In bank 0, b will have the value 100.

In bank 1, b:.* will have the value 00100, since the full address of a bank 0 symbol is the same.

To reinforce, a : always results in a 16-bit address regardless of where it is used.

The formal calculation is $(bankno < < 12) + symbol-value$.

As a convenience, the form *integer:bankno* can be used. The same as above, it is a shorthand for $(bankno < < 12) + integer$, e.g. 100:3.

Code using banks and cross-bank references must understand the standard PDP-1 extended memory mode access rules.

As an example, the *jda* instruction does not work with extended memory. It uses the indirect bit as part of the opcode and will not actually indirect. A macro that implements a working *jda* for extended memory, *farjda()* is provided in the <memory.ah> include file along with other macros useful for cross-bank referencing.

The **macro** and **macro1** compilers do not support extended memory, so code using banks will generate code that isn't actually usable, but it will be annotated to show where banks were switched.

Start or stop

The start or stop directive must be the last statement in a program, and this is enforced. *Start* tells the loader where to begin running the program. The start address can be a numeric address or a location symbol. The start address can also be in any bank and can be a shared location symbol.

The *stop* directive tells the loader to halt instead of starting the program. Additional tapes can be loaded via read-in.

Examples are:

```
start 100
start begin
start 20100 // start at location 100 in bank 2
start begin:3 // start at the location of *begin* in bank 3
stop
```

Export and import

The *export* directive is used to indicate which symbols are to be shared with other programs:

```
export symbol, symbol, ....
```

The directive can be repeated as often as desired. Each symbol listed is marked for export. This automatically asserts the -s command flag if it has not been given.

The symbol is not qualified with a bank reference, its bank is the current bank when the export is done. Exporting a symbol will define it in the current bank if it isn't already defined, and as usual it is an error if it is never resolved.

```
bank 3
export a, b, c
.
.
.
bank 1
export x, y, a
```

Note that symbol 'a' exists separately in two different banks.

The *import* directive names a file to import symbols from, and has the same form as the *#include* directive for **c++**. It

can also be repeated as needed.

```
import "foo.sym"  
import <libx.sym>
```

Just as for include files, the bracketed form looks in the system include directory for the file.

The exported symbols in the file will be created in the global symbol table for the bank they were exported from and will be resolved to the address they were exported with.

It is an error to duplicate a symbol of the same name in the same bank, it is treated as if it were defined in the bank. This means any import must be done before a symbol is resolved.

Imported symbols follow the same rules as cross-bank references; the address will always be 16 bits except in the bank where the symbol was defined, in which case it will be 12 bits.

No code is generated for imported symbols, they are just references. Note that the symbol file will contain all global symbols from the exporting code, only those marked as exported will be imported.

They are accessed using the normal inter-bank notation, *sym:n*.

Special directives

There are two special directives that are not of general use.

One is *%forcelocal*, which is a terrible hack to allow **mactoa**m1 to convert macro-style defines to **cpp** style. It causes any location symbol, even if not marked as local, seen in a local scope to be assumed local. If that same symbol is then used outside local scopes, it is made non-local.

The second is the **cpp**-inserted line and file marker, used to keep the current file name and line number correct. It will always be on a line by itself and is of the form:

```
# number "filename"
```

It is recommended that neither be used directly.

An example

This is a simple example that shows various aspects of multi-bank operation.

Simple am1 demo using multiple banks

```
// The loader will have left extended memory enabled.
// The initial bank is 0.
#include <memory.ah>

100/
    cla
loop,  lio val; lio val; lio val      // just a delay
    add val
    sma
    jmp loop
    farjda(update:*) // a macro in memory.ah
    jmp loop

val,  1          // initially, we increment

bank 1
100/
update, 0          // we jump here from bank 0
    dac rtn
    lac i [val:0]  // change the sign of the operand
    cma
    fardac(val:0) // another macro
    lac update
    jmp i rtn
rtn,  0
constants

start 100
```

Reserved symbols

This is a complete list of all reserved symbols. Reserved symbols cannot be redefined as location symbols within a program. However, **cpp** can redefine them via the *#define* directive, since it runs first.

Miscellaneous	Value
1s	01
2s	03
3s	07
4s	017
5s	037
6s	077
7s	0177
8s	0377
9s	0777
i	010000
C	004000

Opcodes	Value
and	0020000
ior	0040000
xor	0060000
xct	0100000
jfd	0120000
cal	0160000
jda	0170000
lac	0200000
lio	0220000
dac	0240000
dap	0260000
dip	0300000
dio	0320000
dzm	0340000
add	0400000
sub	0420000
idx	0440000
isp	0460000
sad	0500000
sas	0520000
mus	0540000
dis	0560000
mul	0540000
div	0560000
jmp	0600000
jsp	0620000
law	0700000
skp	0640000
spi	0642000
szo	0641000

Opcodes	Value
sma	0640400
spa	0640200
sza	0640100
szf	0640000
szs	0640000
ral	0661000
rar	0671000
ril	0662000
rir	0672000
rcl	0663000
rcr	0673000
sal	0665000
sar	0675000
sil	0666000
sir	0676000
scl	0667000
scr	0677000
nop	0760000
opr	0760000
cli	0764000
lat	0762200
cma	0761000
hlt	0760400
cla	0760200
lap	0760300
clf	0760000
stf	0760010

1D instructions	Value
lia	0760020
lai	0760040
lsw	0760060
swp	0760060
cmi	0764000
sni	0644000
szl	0654060

Various IOTs	Value
iot	0720000
ioh	0720000
tyi	0720004
rrb	0720030
cks	0720033
rpa	0730001
rpb	0730002
tyo	0730003
ppa	0730005
ppb	0730006
dpy	0730007
dpyc	0740007
sdb	0722007
lem	0720074
eem	0724074

SBS ops	Value
esm	0720055
lsm	0720054
cbs	0720056

SBS16 ops	Value
dsc	0720050
asc	0720051
isb	0720052
cac	0720053

Keywords
%forcelocal
addlocal
ascii
bank
char
constants
decimal
endlocal
export
flexo
import
local
octal
start
stop
table
text
var
variables